F1HUB

Technical Documentation — Intermodular Project

Multiplatform Application Development (DAM)

Academic Year 2025 / 2026

Application: F1HUB – Social network based on F1

Author: Lucas González Parada

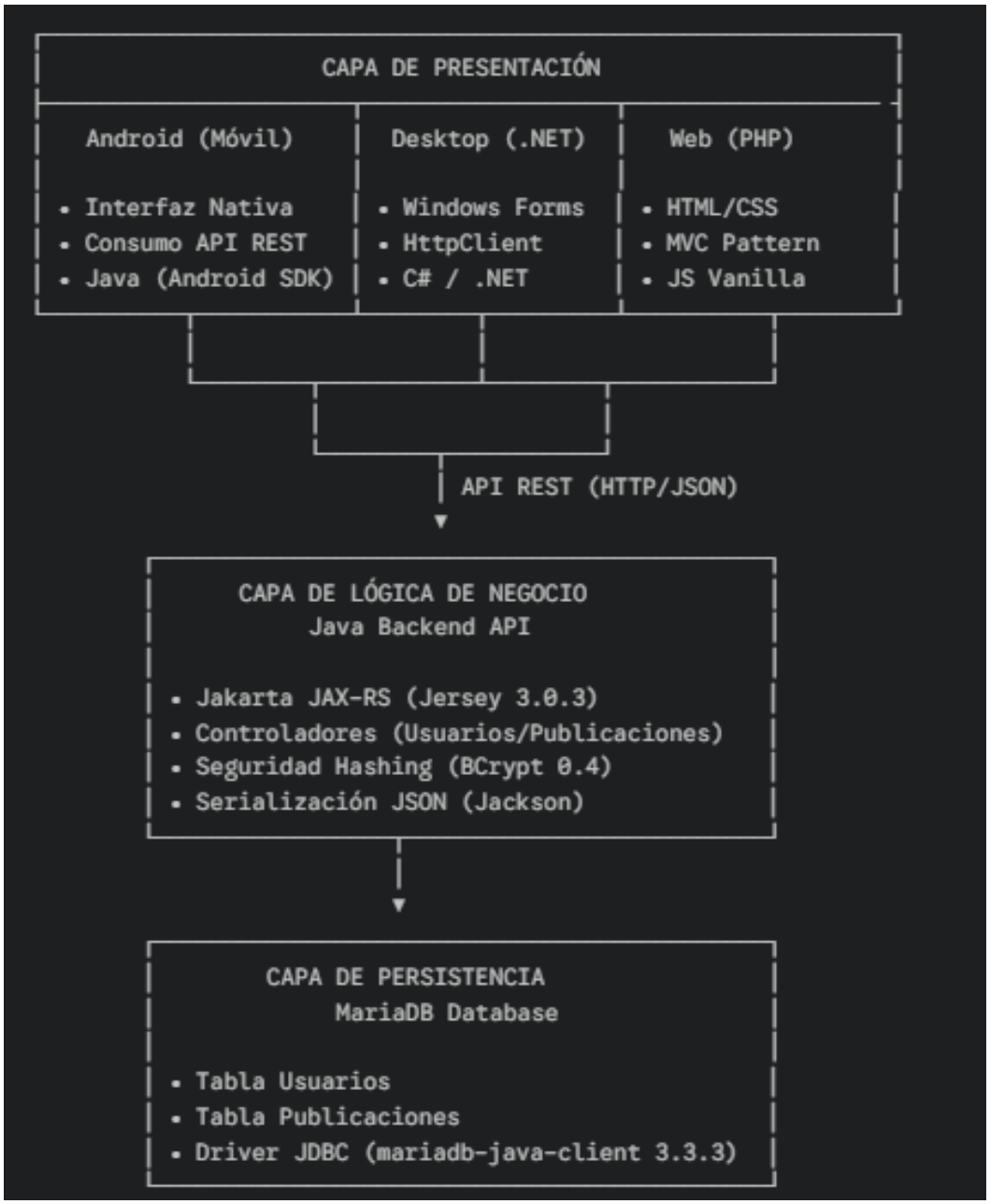
Date : March 2026

Table of Contents

1. System Architecture
2. API
   2.1 General Description
   2.2 Endpoint List
   2.3 Request and Response Examples
   2.4 Status Codes and Error Handling
3. Database
   3.1 General Description
   3.2 Entity-Relationship Diagram
   3.3 Table Descriptions
   3.4 Integrity Rules and Constraints
4. Web Application
   4.1 Technologies Used
   4.2 Project Structure
   4.3 API Connection Flow
5. Mobile Application
   5.1 Technologies Used
   5.2 Project Structure
   5.3 API Connection
   5.4 Main Features
6. Desktop Application
   6.1 Technologies Used
   6.2 Project Structure
   6.3 API Connection
   6.4 Main Screens
7. Security
   7.1 Security Mechanisms Implemented
   7.2 User and Permission Management
   7.3 Best Practices and Additional Measures
8. Deployment
   8.1 Prerequisites and Installation
   8.2 Steps for Local Execution
   8.3 Environment Configuration
10. Usage and Maintenance Guide
   10.1 Running Locally
   10.2 Adding New Features
   10.3 Common Issues
11. Future Improvements
   11.1 Planned Features
   11.2 New User Features
   11.3 Technical Optimisations

1. System Architecture

The F1HUB system is composed of three client applications (Web, Mobile, and Desktop) that communicate with a centralised RESTful API, which in turn manages access to a shared MariaDB database.



1.1 General Overview

All three client applications connect independently to the same backend (Java/Jakarta EE API). No application accesses the database directly; all business logic and data persistence is routed through the API.

1.2 Main Technologies by Layer

Layer	Technology	Description
API Backend	Java / Jakarta EE (Jersey)	RESTful API acting as intermediary between clients and database.
Database	MariaDB	Relational engine with the f1hub schema.
Web Application	PHP 8 + Apache (XAMPP)	MVC pattern with cURL calls to the API.
Mobile Application	Java/Kotlin + Android SDK	Native Android app (API level 24+).
Desktop Application	C# + .NET Framework (WinForms)	Windows client using HttpClient.

2. API

2.1 General Description

The RESTful API is developed in Java using the Jakarta EE (Jersey) framework. Its main role is to act as an intermediary between the client applications and the MariaDB database.

All communications use JSON format (application/json) through the standard HTTP methods (GET, POST, PUT, DELETE).

Base URL (Local Development):
http://localhost:8080/f1hub/rest

2.2 Endpoint List

User Management (/usuarios)

Endpoint	Method	Description
/usuarios/registro	POST	Creates a new user, encrypting their password.
/usuarios/inicioSesion	POST	Validates credentials and returns the user

Endpoint	Method	Description
		profile.
/usuarios/editar	PUT	Updates the personal data of an existing user.

Post Management (/publicaciones)

Endpoint	Method	Description
/publicaciones/subir	POST	Creates a new post associated with a user.
/publicaciones/todas	GET	Retrieves the general feed (all posts ordered by date).
/publicaciones/{idUsuario}	GET	Retrieves the list of posts for a specific user.
/publicaciones/{id}	DELETE	Deletes a post by its ID.

2.3 Request and Response Examples

2.3.1 User Registration

Request:

```
POST /rest/usuarios/registro HTTP/1.1
Content-Type: application/json

{
  "nombre": "Fernando",
  "apellidos": "Alonso",
  "nombreUsuario": "MagicAlonso",
  "passwordHash": "123456",
  "email": "fernando@f1.com",
  "genero": "Masculino",
  "fechaNacimiento": "1981-07-29"
}
```

Successful Response (201 Created):

```
1  Usuario insertado correctamente
```

2.3.2 Login

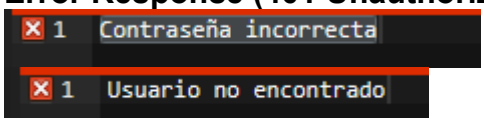
Request:

```
POST /rest/usuarios/inicioSesion HTTP/1.1
Content-Type: application/json

{
  "nombreUsuario": "MagicAlonso",
  "passwordHash": "123456"
}
```

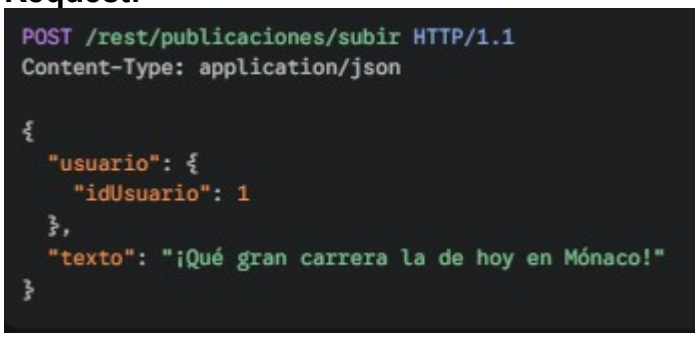
Successful Response (200 OK):

Error Response (401 Unauthorized):

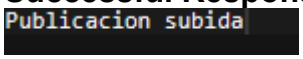


2.3.3 Create a Post

Request:



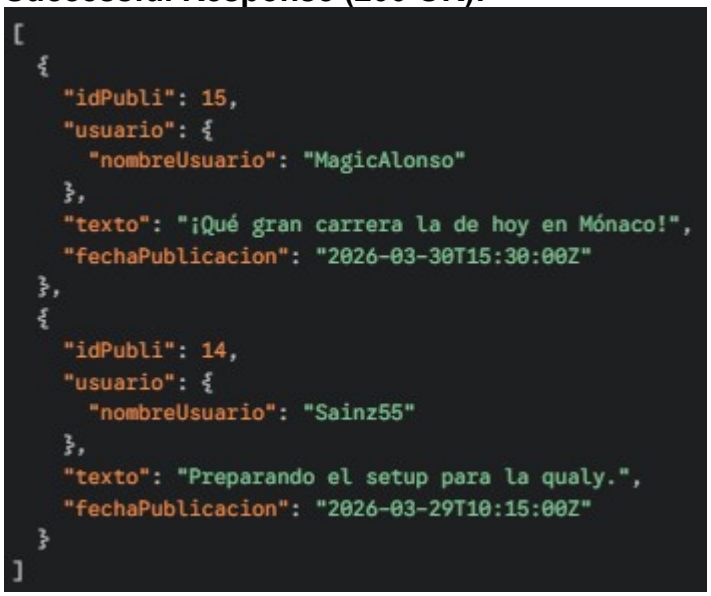
Successful Response (200 OK):



2.3.4 Get All Posts (Feed)

Request:

Successful Response (200 OK):



2.3.5 Delete a Post

Request:

```
DELETE /rest/publicaciones/15 HTTP/1.1
```

Successful Response (200 OK):

```
{
  "mensaje": "Publicacion eliminada"
}
```

2.4 Status Codes and Error Handling

Code	Meaning	Typical cause in F1HUB
200	OK	Request processed and data returned successfully.
201	Created	User registered successfully in the database.
400	Bad Request	Mandatory fields missing in the JSON (e.g. null credentials).
401	Unauthorized	The password provided during login is incorrect.
404	Not Found	The username does not exist in the database.
500	Internal Server Error	MariaDB connection failure, SQL syntax error, or JDBC driver not found.

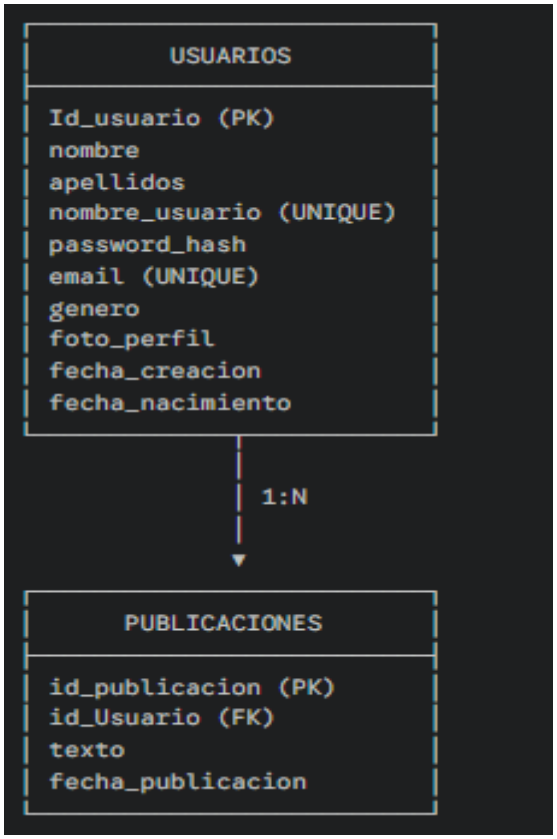
3. Database

3.1 General Description

The system uses MariaDB as its relational database engine. The database, named f1hub, is designed to guarantee referential integrity and optimise queries for the post feed.

3.2 Entity-Relationship Diagram

The main data model is based on a One-to-Many (1:N) relationship between users and their posts:



3.3 Table Descriptions

3.3.1 Users Table

Field	Data Type	Null	Attributes / Extra	Description
Id_usuario	int(11)	No	PK, AUTO_INCREMENT	Unique user identifier.
nombre	varchar(100)	No		User's real first name.
apellidos	varchar(100)	No		User's surnames.
nombre_usuario	varchar(50)	No	UNIQUE	Public nickname. Cannot be repeated.
password_hash	varchar(255)	No		Password encrypted using the BCrypt algorithm.
email	varchar(100)	No	UNIQUE	Registration email. Cannot be repeated.
genero	varchar(20)	Yes	DEFAULT NULL	User's gender.

Field	Data Type	Null	Attributes / Extra	Description
foto_perfil	longblob	Yes	DEFAULT NULL	Profile picture stored in binary format.
fecha_creacion	timestamp	No	DEFAULT current_timestamp()	Exact date and time the account was registered.
fecha_nacimiento	date	Yes	DEFAULT NULL	Date of birth for age calculation.

3.3.2 Posts Table

Field	Data Type	Null	Attributes / Extra	Description
id_publicacion	int(11)	No	PK, AUTO_INCREMENT	Unique post identifier.
id_Usuario	int(11)	No	FK	Reference to the author (Usuarios.Id_usuario).
texto	text	No		Post content.
fecha_publicacion	timestamp	No	DEFAULT current_timestamp()	Automatic timestamp set when the post is inserted.

3.4 Integrity Rules and Constraints

1. Unique Keys: Unique indexes have been set on the nombre_usuario and email fields in the Users table to prevent duplicate records at the database level, providing an additional security barrier on top of the API validations.
2. Automatic Timestamps: The fecha_creacion (Users) and fecha_publicacion (Posts) fields use the current_timestamp() function. This delegates timestamp responsibility to the MariaDB engine, avoiding time-zone discrepancies between different clients (C#, Android, Web).
3. Referential Integrity (Foreign Key): The id_Usuario field in Posts is strictly linked to Id_usuario in the Users table. If a user is deleted, the system must handle cascade deletion (CASCADE) of their posts to avoid orphaned records.

4. Web Application

4.1 Technologies Used

Component	Technology	Technical Details
Backend / Logic	PHP 8	Implements the MVC architectural pattern.
Frontend / Design	HTML5 and CSS3	Native markup and styles, no heavy frameworks.
Interactivity	Vanilla JavaScript	Client-side scripts (e.g. slider.js).
Web Server	Apache	Deployed via the XAMPP local environment.
Communications	cURL (PHP Library)	Sends HTTP requests to the Java REST API.

4.2 Project Structure (MVC Pattern)

Directory / File	Role in the System	Main Content
index.php	Entry Point	Acts as the main router, receiving the action variable.
controladores/	Controller	ControladorPublicaciones.php, ControladorInicioSesion.php, etc.
modelos/	Model	Classes that map JSON data: Publicaciones.php, Usuario.php.
vistas/	View	Files with the final interface: Muro.php, Perfil.php, Registro.php.
public/	Static Resources	Stylesheets (css/Muro.css), scripts (js/slider.js), and images.

4.3 API Connection Flow

Step	Communication Phase	Technical Description
1	Capture and Formatting	The Controller receives the user interaction ($_POST) and converts the data to JSON format using json_encode().
2	Request Dispatch	Via cURL, PHP sends an HTTP request (GET, POST, PUT, DELETE) to the Java endpoint, setting the Content-Type: application/json header.

Step	Communication Phase	Technical Description
3	Data Reception	The Java API processes the logic and returns a status code (e.g. 200 OK) along with a JSON response body.
4	State Management	PHP decodes the response with json_decode(). On a successful login, it stores the profile in the global $_SESSION['usuario'] variable.
5	Final Rendering	The Controller injects the retrieved data (e.g. the feed list) into the View, which generates the final HTML seen by the user.

5. Mobile Application

5.1 Technologies Used

Component	Technology	Technical Details
Core Language	Java/Kotlin	Native development using the Android SDK.
Environment (IDE)	Android Studio	Project management and build via Gradle.
SDK Version	Android 7.0	API Level 24
UI Design	XML / Material Design	Modern components and adaptive layouts.
Key Libraries	BCrypt	Local security (BCrypt).

5.2 Project Structure

Package / Directory	Role in the System	Main Components
activities/	Screen Controllers	MainActivity.java, InicioSesion.java, Registro.java, RegistroPassword.java, SubidaPublicaciones.java and EditarPerfil.java.
fragments/	Interface Modules	HomeFragment.java, PerfilFragment.java, SalirFragment.java.
adapters/	List Management	AdaptadorPublicaciones.java for inflating the RecyclerView.
models/	Data Objects	Usuario.java and Publicaciones.java.
api/	Network Service	ApiRest.java

Package / Directory	Role in the System	Main Components
res/layout/	Visual Layouts	XML files defining the user interface screens.

5.3 API Connection

Phase	Operation	Technical Description
1	HTTP Request	The mobile client initiates a request (POST for login/registration, GET for the feed) to the backend URL.
2	Serialisation	Data is sent and received in JSON format, automatically mapped to Java objects via processing libraries.
3	Storage	After a successful login, the user data is kept in memory or cache to personalise the Profile and Home Fragments.
4	Update	The AdaptadorPublicaciones receives the post list from the API and dynamically updates the feed in the HomeFragment.

5.4 Main Features

- Authentication: Registration and login system connected in real time to the centralised database.
- News Feed: Display of the latest F1HUB community posts via a dynamic list.
- Profile Management: Dedicated screen to view and edit the current user's information.
- Post Creation: Form to write and submit new posts to the feed.

6. Desktop Application

6.1 Technologies Used

Component	Technology	Technical Details
Language	C#	Modern object-oriented syntax.
Framework	.NET Framework	Stable platform for Windows applications.
Interface (UI)	Windows Forms	Native window system (WinForms).
API Communication	HttpClient	.NET class for asynchronous HTTP requests.

Component	Technology	Technical Details
JSON Handling	JavaScriptSerializer	Serialisation and deserialisation of API data.

6.2 Project Structure

Directory / File	Role in the System	Key Components
Forms/Auth/	Access Forms	FormInicioSesion.cs and FormRegistro.cs.
Forms/Main/	Main Interface	FormPrincipal.cs (Feed) and FormPerfil.cs.
Models/	Data Classes	Usuario.cs and Publicacion.cs (business objects).
Services/	Service Layer	ApiClient.cs (centralises all API calls).
App.config	Configuration	Stores parameters such as the API base URL.
Program.cs	Entry Point	Application startup logic.

6.3 API Connection

Phase	Operation	Technical Description
1	Encapsulation	Forms instantiate the ApiClient to request or submit data.
2	Asynchrony	Async methods are used to send requests to the backend without freezing the user window.
3	Object Mapping	The JavaScriptSerializer transforms received JSON into Usuario objects or lists of Publicacion.
4	Error Handling	Status codes (401, 404, 500) are handled to display alert messages to the user on failure.

6.4 Main Screens

- Login and Registration: Dedicated windows for access management, validating fields before sending them to the API.
- Main Panel (Feed): Central interface where all F1HUB posts are loaded and displayed in an organised manner.
- User Profile: Form for viewing and updating the information of the currently logged-in driver/fan.

7. Security

7.1 Security Mechanisms Implemented

Mechanism	Technical Description	Objective
Password Hashing	Use of the jbcrypt library (v0.4) with a cost factor of 10.	Avoid storing passwords in plain text, protecting accounts against potential data leaks.
Prepared Statements	Use of PreparedStatement in all JDBC interactions in the Java API.	Prevention of SQL Injection attacks, ensuring input parameters are treated as data, not executable code.
Identity Validation	Verification via the BCrypt.checkpw() method during the login process.	Ensure that only the account owner can access the protected functions of the applications.
Uniqueness Constraint	UNIQUE indexes in the MariaDB database for email and nombre_usuario.	Prevent identity spoofing and duplicate records at the database engine level.

7.2 User and Permission Management

Access management is centralised in the Java API, which acts as the sole gatekeeper of resources:

- Stateless Authentication: The API validates credentials on every access attempt, returning the full User object only if the password matches the stored hash.
- Session Control in Web: The PHP application uses the global $_SESSION variable to maintain the user's identity after login, redirecting to the login form if no active session is detected.
- Sensitive Data Protection: The data models in C#, Java, and PHP are designed not to expose the password hash in public views, limiting shared information to profile data (name, surnames, photo).

7.3 Best Practices and Additional Measures

4. Character Escaping: In the Web view (PHP), htmlspecialchars() is used to render usernames and post content, mitigating Cross-Site Scripting (XSS) risks.
5. Connection Closing: Use of try-with-resources blocks in Java to ensure that MariaDB connections are automatically closed, preventing memory leaks and potential Denial of Service (DoS).

6. Generic Error Handling: The API returns controlled error messages ("User not found", "Incorrect password") instead of system stack traces, avoiding giving attackers clues about the internal structure of the database.

8. Deployment

8.1 Prerequisites and Installation

Component	Required Software	Purpose
Database and Web	XAMPP (Apache + MariaDB)	PHP web server and database engine.
API Backend	Apache Tomcat 10 / Eclipse / VS Code	Servlet container to run the Java .war file.
Java Environment	JDK 1.8 (Java 8)	Development kit needed to compile and run the API.
Desktop App	Visual Studio 2019 or later	IDE to compile the C# .sln solution.
Mobile App	Android Studio	IDE to compile and deploy the APK to an emulator or device.

8.2 Steps for Local Execution

Deployment must follow a logical order to ensure clients can connect to the server:

1. Database Setup:
- Start the MySQL module in the XAMPP Control Panel.
- Open phpMyAdmin and create the f1hub database.
- Import the SQL file containing the Users and Posts tables.

2. API Deployment (Backend):
- Open the Maven project in the IDE (Eclipse or VS Code).
- Run Maven Install to generate the f1hub.war file.
- Deploy the project to the Apache Tomcat server.
- Verify the connection at: http://localhost:8080/f1hub/rest/usuarios/registro.

3. Web Application Deployment:
- Copy the PHP project folder into the htdocs directory of XAMPP.
- Start the Apache module in XAMPP.
- Access via browser at: http://localhost/f1hub_web/index.php.

4. Running Native Clients:
- Desktop: Open F1HUB.sln in Visual Studio and press F5 (Debug).
- Mobile: Open the project in Android Studio, sync with Gradle, and run on an emulator with API 24+.

8.3 Environment Configuration

For the system to work across different networks, the server IP address must be taken into account:

- Local Environment: All applications point to localhost or 127.0.0.1.
- Mobile Environment (Emulator): Android uses the special IP 10.0.2.2 to refer to the host PC's localhost.
- Real Environment: Replace "localhost" with the computer's private IP address (e.g. 192.168.1.XX) in the files ApiClient.cs (C#), ApiRest.java (Android), and in the PHP controllers.

10. Usage and Maintenance Guide

10.1 Running Locally

To start up the complete system in a development environment, follow this workflow:

1. Start Data Services:

- Open the XAMPP Control Panel and activate the Apache and MySQL modules.
- Verify that port 3306 is free for MariaDB.

2. Server Deployment (API):

- In Eclipse or VS Code, ensure the Tomcat 10 server is configured with Java version 1.8.
- Right-click on the f1hub project > Run As > Run on Server.
- The API is ready when the console shows: "Deployment of web application archive has finished".

3. Accessing the Applications:

- Web: Navigate to http://localhost/f1hub/ in any modern browser.
- Desktop: Run the F1HUB.exe file generated in the bin/Debug folder of the C# solution.
- Mobile: Launch the emulator from Android Studio and run the app (Shift + F10).

10.2 Adding New Features

The system has been designed in a modular fashion to facilitate expansion:

Adding new fields to the Database:

7. Modify the table in phpMyAdmin.
8. Update the POJO (Java class) in the backend and the models in C#, Android, and PHP.
9. Adjust the SQL queries in the Java API controllers.

Creating a new Endpoint in the API:

10. Locate the ApiRest.java or ApiRestPublicaciones.java class.

11. Define the new method with the corresponding annotations (@GET, @POST, etc.) and path (@Path).
12. Restart the Tomcat server for the changes to take effect.

Modifying the Web interface:

13. Visual styles are centralised in the public/css/ folder.
14. Navigation logic is managed in the index.php file via the "action" variable.

10.3 Common Issues

Issue	Probable Cause	Suggested Solution
Error 404 Not Found	Project name missing from the URL or Tomcat has not started.	Make sure to include /f1hub/ in the path and verify the server is green in the IDE.
Error 415 Unsupported Media Type	The request has not been specified as JSON.	In the client (C#, Android, or Thunder Client), set the Content-Type header to application/json.
Error 500 Internal Server Error	MariaDB connection failure or SQL syntax error.	Check that MySQL is active in XAMPP and that the user/password in ApiRest.java are correct.
Connection error on Android	The emulator cannot reach the PC's localhost.	Change "localhost" to "10.0.2.2" in the API configuration of the mobile project.

11. Future Improvements

11.1 Planned Features

The F1HUB project has great scalability potential. The following areas for improvement have been identified for future versions:

- Notification System: Implement push notifications in the mobile app and visual alerts in the web and desktop to inform users when they receive interactions on their posts.
- Advanced Social Interaction: Add a "Like" system (reactions) and the ability to reply to posts (comment threads) to encourage debate among fans.
- Multimedia Content: Optimise the API and applications to allow the upload of short "paddock" videos and high-resolution image galleries.
- Real-Time Data Integration: Connect to an external Formula 1 API to display race results, lap times, and the drivers' and constructors' championship standings directly in a dedicated widget.

11.2 New User Features

- Fan Geolocation: An optional feature to view on an interactive map where other fans are located during Grand Prix weekends.
- Account Verification: A verified profile system for journalists, mechanics, or real drivers who join the platform.

11.3 Technical Optimisations

To improve efficiency and user experience, the following optimisations are proposed:

- WebSocket Implementation: Replace manual refresh with a persistent connection via WebSockets (e.g. Socket.io or Java WebSockets) so that new posts appear in the feed instantly without reloading the page.
- JWT Security: Evolve the current authentication system towards the use of JSON Web Tokens (JWT). This would enable more secure and professional session management, removing the dependency on PHP sessions and making the API easier to scale.